

Q1. Infinite streams of characters, after a new character comes, check if the current stream of characters forms a palindromic string or not

: rearrangement of str is not allowed

a ✓

a b ✗

a b c ✗

a b c b ✗

a b c b a ✓

Basic $\Rightarrow$ check the new str for palindrome after every new character

palindrome $\rightarrow S == \text{rev}(S) \rightarrow \checkmark$

str $\Rightarrow$ $\begin{matrix} 2 & 1 & 0 \\ a & b & c \end{matrix} \Rightarrow a \times p^2 + b \times p^1 + c \times p^0$

rev(str) $\Rightarrow$ $\begin{matrix} 2 & 1 & 0 \\ c & b & a \end{matrix} \Rightarrow \underbrace{[c \times p^2 + b \times p^1 + a \times p^0]}_{\checkmark}$

str $\Rightarrow$ $\begin{matrix} a & b & a \end{matrix} \Rightarrow \underbrace{a \times p^2 + b \times p^1 + a \times p^0}_{\checkmark}$

rev(str) $\Rightarrow$ $\begin{matrix} a & b & a \end{matrix} \Rightarrow \underbrace{a \times p^2 + b \times p^1 + a \times p^0}_{\checkmark}$

	(forward hash) FH	(reverse hash) RH	palindrome
a	$a \times p^0$	$a \times p^0$	✓
a b	$a \times p^1 + b \times p^0$ $\rightarrow (prev) \times p + b$	$b a$ $b \times p^1 + a \times p^0$ $\rightarrow b \times p^{(len-1)} + (prev)$	X
a b a	$a \times p^2 + b \times p^1 + a \times p^0$ $\rightarrow (prev) \times p + a$	$a \times p^2 + b \times p^1 + a \times p^0$ $\rightarrow a \times p^2 + (prev)$	✓

$$\begin{aligned}
 FH &= (FH) \times p + \text{new element} \\
 RH &= (\text{new element}) \times p^{(len-1)} + (RH)
 \end{aligned}$$

$$(FH == RH) \rightarrow \underline{\text{palindrome}}$$

if, n characters are coming:

$$\begin{aligned}
 TC &\Rightarrow O(N) \\
 SC &\Rightarrow O(1)
 \end{aligned}$$

Q2. Find length of longest substring which contains all unique / distinct characters.

ex $\Rightarrow$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	a	e	b	c	a	b	g	e	b	@	#	g	b	k	d	b	#

(5)
(6)
(4)

Ans $\Rightarrow$ 6

Brute $\Rightarrow$ Consider all possible substrings $\rightarrow$ Hashset
 $TC \Rightarrow O(N^2)$ $SC \Rightarrow O(N)$

min length max length
 $\downarrow$ N
 Binary search mid
 $\downarrow$
check substrings of size mid

$TC \Rightarrow O(N \log N)$
 $SC \Rightarrow O(N) \rightarrow$ hashset

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
a e b c a b g e b @ # g b k d b #

ans $\Rightarrow$ ~~K~~ ~~A~~ ~~B~~ ~~X~~ ~~\$~~ (6)

HashMap

a	4	g	11	d = 14
e	7	@	9	
b	15	#	16	
c	3	k	13	

start of solving

↓

$l = 0, r = 0$

while ($r < N$) {

if ($s[r]$ is already in HM) {

if ($hm.get(s[r]) > s$) {

$hm(s[r]) + 1$, update $hm, s[r]$

} else {

$hm.add(s[r])$

$r++$

}

$ans = \max(ans, (r - l + 1))$

TC $\Rightarrow O(N)$

SC $\Rightarrow O(N)$

}

Q3. Count all permutations of A in B as a subseq. (100%)

$A = a \ b \ c$

$B = a \ b \ c \ b \ a \ c \ a \ b \ c$

Ans = 5

Approach → generate all permutations of A
and how many times those permutations
exist in B

$A = a \ c \ a$

$B = a \ | \ c \ a \ a$

↓

freq → window

a c
2 1

freq

a c
2 1

freq → a

a c
2 1

- i) generate freq map A
- ii) generate freq map of all windows of length (A)
- iii) compare the freq map of window & freq map of A

ii) $O(k) \leftarrow \text{freq map } A$

$\Rightarrow$ ii) $O(k) \leftarrow \text{freq map of 1 window}$
no. of windows $\Rightarrow N-k+1$
total time freq map $\Rightarrow O(k(N-k+1))$

iii) comparing the maps:-
 $O(26)$

$A = abc$

$B = abaca$

$B = a \quad b \quad [a \quad c \quad a]$

a	b	c
2	0	1

freqmap[removed char] --

freqmap[new char] ++ $\leftarrow$ increase freq
or, add it

$A = abc$

freqmap

a	b	c
1	1	1

* Sliding window to generate freq map $\rightarrow \tau \Rightarrow O(N-k)$

$$\begin{aligned}
 & \text{i) freq } A \rightarrow O(k) \\
 & \text{ii) freq window} \rightarrow O(N-k) \\
 & \text{iii) 26 comparisons} \\
 & \quad \downarrow \\
 & \text{Overall} \rightarrow T.C = O(26N) \\
 & \quad S.C = O(26)
 \end{aligned}
 \quad \left. \begin{array}{l} \text{i) } \\ \text{ii) } \end{array} \right\} \rightarrow O(N)$$

max characters $\rightarrow$ 26

$A = a \ b \ c$

$B = a \ b \ c \ b \ a \ c \ a \ b \ c$

curr-freq-map

$a \ 1$
 $b \ 1$
 $c \ 1$

need = 3

$\neq$
 $\neq$
 $\neq$
 0

$\frac{\text{req freq}}{\text{freq } A}$

a	b	c
1	1	1

 need = 3

$$\text{curr freq}(b) > \text{req-freq}(b)$$

$$\text{curr freq}(a) < \text{req-freq}(a)$$

a = a b c

s = a a b b c a b c

// first window

```
for (i = 0; i < k; i++) {
```

```
    if (s[i] present in window-map) {
```

```
        windowmap(s[i]) --
```

```
        if (s[i] == 0)
```

```
            need --
```

```
    }
```

```
    if (need == 0) count++
```

a a b

a a b c b b d

↑

req freq

a	b	c
1	1	1

curr-freq

window-map

a	b	c
x	x	1
0	0	-1

a	b
2	1
	0
	-1

TC $\approx O(N)$

SC $\approx O(N)$

Adde

Q4. Find the shortest substring in A that contain all the characters of B.

A = A D O B E C O D E B A N C

B = A B C

A = A D O B E C O D E B A N C

B = A B C

A	B	C
✓	✓	✓
0	0	0

A D O B E C O D E B A N C

ans = 6 4 ←

TC	$\approx O(N)$
SC	$\approx O(K)$